

## 프로젝트 제목: [ 새 키우기 ]

- 팀명: RLCC
- 구성원: 유이레, 이준규, 최승민, 최연채
- 깃허브 링크: [https://github.com/jun9yu-527/BirdRaise/tree/ver\\_1.3](https://github.com/jun9yu-527/BirdRaise/tree/ver_1.3)
- 동영상 링크: [https://www.youtube.com/watch?v=R\\_kRyPf1t8g](https://www.youtube.com/watch?v=R_kRyPf1t8g)

### 1. 프로젝트 개요

- 작품주제 선정 배경: 길에서 알을 줍는 스토리로 시작하여, 8주 동안 알부터 성체까지 새를 정성껏 돌보아 높은 만족도를 기록하고 그에 따른 엔딩을 확인하는 육성 시뮬레이션 게임입니다.
- 개발 환경 설명
  - 개발환경: Microsoft Visual Studio 2022
  - 언어표준: C++17
  - 운영체제: Windows Console(CMD/PowerShell)
  - 구성: UTF-8 인코딩 소스 및 실행 환경 구축 (/utf-8 옵션 적용)
- 벤치마킹 자료

|                                                                                     |                                                                                      |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <p>- 다마고치<br/>(휴대용 디지털 반려동물 육성 게임)</p>                                              | <p>- 인생게임<br/>(캐릭터의 일생을 '달리기' 형식으로 표현한 게임)</p>                                       |
|  |  |

---

## 2. 개발 프로그램 설명

### 2.1 클래스 다이어그램 및 파일 구성

프로그램은 5개의 주요 클래스로 구성되어 있습니다.

- **GameEntity (새 객체 클래스)**

- **주요 속성:** 이름, 핵심 수치(포만감, 청결도, 훈련도, 스트레스), 성장 단계(알, 유아기, 유년기, 청소년기, 성체), 생존 여부.
- **주요 메서드:**
  - 행동에 따른 수치 변화를 적용하고 범위를 제한하는 `update_stats`.
  - 질병/반항/사망/도주 상태를 체크하는 `check_status`.
  - 실시간 평균 만족도를 계산하는 `get_satisfaction`.
  - 1주차 알 단계에서 모든 행동을 "방치하기"로 선택 시 `abandonedEgg = true`로 설정하여 화석 엔딩 확인하는 `recordEggAction(bool isIgnore)`.
  - 스트레스 50 이상 시 20% 확률로 반항 이벤트를 발동하여 스트레스 +10 패널티 부여하는 `is_rebellious()`.
- **만족도 산출 공식:** (포만감 + 청결도 + 훈련도 + (100 - 스트레스)) ÷ 4. 엔딩 분기의 핵심 지표로 매 턴 실시간 갱신되며 0~100 범위로 클램프.

- **Time (게임 흐름 제어 클래스)**

- **주요 속성:** 8주차까지의 시간 흐름을 기록, 주차 내 남은 행동 횟수 기록, 마지막 성장 처리 (중복 성장 방지에 사용)
- **주요 메서드:**
  - 전체 게임 루프를 관리하는 `run_game_loop`
  - 플레이어 선택에 따른 결과값 출력 및 스탯을 반영하는 `process_action`
  - 행동 횟수 차감 및 시간/주차 변경 로직인 `next_turn`
  - 추가로 매주 포만감 -10, 청결도 -5, 스트레스 +5를 자동 적용하는

apply\_weekly\_passive와, last\_grown\_week로 중복 성장을 방지하는  
check\_growth.

-알 단계(1주차)에서는 사망 판정을 수행하지 않는 예외 처리도 포함.

- **ActionManager (행동 및 선택지 클래스)**

- **주요 데이터 및 속성:** 성장 단계별 행동 메뉴, 선택지별 수치 변화값 테이블

- **주요 메서드:**

-현재 성장 단계에 맞는 메뉴를 반환하는 get\_available\_actions

-훈련이나 먹이 급여 시 확률적으로 발생하는 아이템 효과를 처리하는  
try\_get\_item (40% 확률로 견과류를 비롯한 아이템 획득 후 set<string> 인벤  
토리에 저장)

-아이템 보유 여부에 따라 먹이 선택지를 동적으로 필터링하는  
get\_sub\_actions 포함

-유아기의 "놀아주기"에서는 "산책하기" 선택지가 자동 제외되는 단계별 예외  
처리 적용.

- **UIRenderer (화면 출력 클래스)**

- **주요 메서드:**

-현재 주차와 상태 정보를 상시 노출하는 display\_status

-단계별 외형 변화를 아스키 아트로 출력하는 draw\_bird

-조건에 따른 엔딩을 묘사하고 출력하는 show\_ending.

- **GameManager (게임 통합 관리 클래스)**

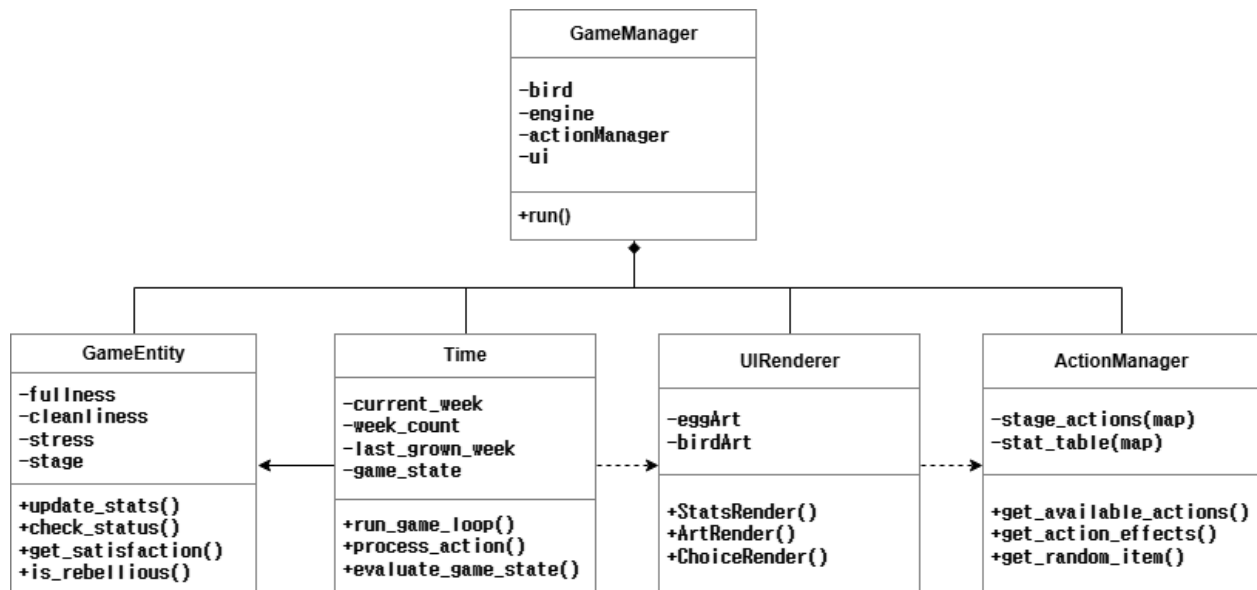
- **주요 속성:** GameEntity, GameEngine, ActionManager, UIRenderer 각 클래스의  
객체

- **주요 메서드:**

-길에서 알을 줍는 시작 스토리 및 이름 설정 등 게임 초기화 로직을 수행하  
는 initializeGame.

-게임 전체 루프를 실행하고 종료 처리를 관리하는 run. calcStageInt() / calcStageName()은 현재 주차를 기반으로 성장 단계를 계산하여 GrowthStage getter 없이 주차 값만으로 단계를 판별.

-processTurn()은 UI 출력 → 메인 행동 선택 → 서브 행동 선택 → 수치 반영 → 반항 체크 → 아이템 처리 → 결과 출력의 순서로 진행.



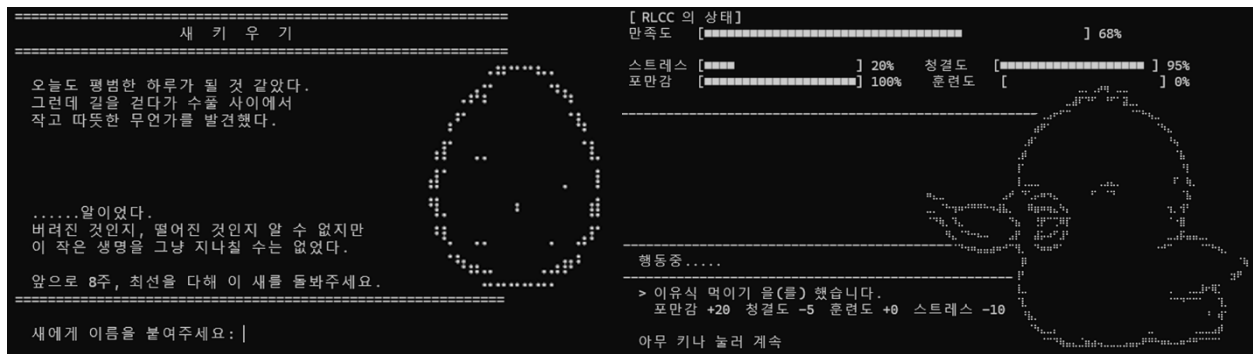
## 2.2 프로젝트 디렉토리 구조

BirdRaise/

```
|—— src/
|   |—— Main.cpp           # 진입점, 콘솔 UTF-8 인코딩 설정
|   |—— GameManager.cpp    # 게임 전체 흐름 관리 (턴 진행, 행동 선택, 엔딩)
|   |—— GameEntity.cpp     # 새 객체 상태 관리 (수치, 성장 단계, 생존 여부)
|   |—— ActionManager.cpp  # 행동 목록, 세부 선택지, 아이템/인벤토리 처리
|   |—— Time.cpp           # 주차 및 행동 횟수 관리, 성장 단계 전환
|   |—— UIRenderer.cpp     # 콘솔 화면 출력 (게이지, 아트, 엔딩)
|—— include/
|   |—— Time.h / GameManager.h / GameEntity.h
|   |—— ActionManager.h / UIRenderer.h
```

## 2.3 UI/UX 설계 (실제 화면 구조 이미지 및 사용자 경험과 인터페이스에 대한 설명)

- **상태 표시바:** drawBar() 메서드가 ■ 문자를 이용한 게이지 바를 출력. 만족도·스트레스·청결도·포만감·훈련도를 항상 화면 상단에 노출
- **SlowPrint 연출:** 스토리 장면은 글자를 한 자씩 딜레이를 주며 출력하는 SlowPrint() 메서드로 몰입감 강화
- **행동 아트 시스템:** 행동을 선택하면 해당 행동에 대응하는 아스키 아트 (ActionArtRender())가 전환되어 시각적 피드백 제공
- **엔딩 아트:** 각 엔딩마다 전용 아스키 아트를 출력하는 EndingArtRender() 메서드 구현.



### 3. 테스트 및 검증 포함, 개발 중 발생한 장애요인과 문제 해결 과정

- **컴파일 오류:** 각 팀원의 PC 환경(Windows 한글 코드페이지 차이 등)에 따라 컴파일러가 소스 코드 내의 한글 문자열을 인식하지 못하는 오류 발생.

- **문제 해결 과정:** 소스 코드에 포함된 한글을 컴파일러가 올바르게 해석하도록 강제 설정했습니다.

비주얼 스튜디오 프로젝트 속성 → C/C++ → 명령줄 → utf-8 입력 후 적용

**결과:** 소스 코드 내의 한글 주석 및 문자열 상수로 인한 컴파일 에러를 해결했습니다.

- Windows 환경에서 콘솔 출력 제어를 위해 시스템 헤더도 추가했습니다.

**추가한 헤더:** #include <windows.h>

**결과:** 콘솔 인코딩을 제어하는 Windows 전용 함수 및 명령어를 안정적으로 사용할 수 있게 되었습니다.

- **런타임 출력 오류:** 프로그램 실행 중 사용자가 입력한 한글 이름이나 프로그램 내 한글 텍스트가 깨져서(깨진 글자나 '?'로 표시) 출력되는 현상 발생.

- **문제 해결 과정:** 사용자가 입력하는 한글과 터미널에 출력되는 한글이 일치하도록 설정했습니다.

**추가한 코드:**

system("chcp 65001"); // 터미널 코드페이지를 UTF-8(65001)로 변경

system("cls"); // 인코딩 변경 시 나타나는 시스템 메시지 삭제

**결과:** 플레이어가 입력한 한글 이름이 깨지지 않고 정상적으로 변수에 저장 및 화면에 출력되는 것을 확인했습니다.

- **날짜 변경 및 행동 횟수 오류:** 1주차에서 2주차로 넘어가는 구간에서 행동 횟수가 5회가 아닌 7회로 변경이 되지 않는 현상 발생.
  - **문제 해결 과정:** Time.h에 daily\_count와 week\_count 두개가 존재 했었지만 daily\_count를 없애고 week\_count로만 다뤄지게 수정하였습니다.
  - 결과:** 하루 행동 횟수가 7회로 바뀌던 오류가 5회로 수정 되었습니다.
- **히든/배드 엔딩 조건 수정:** 초기 기획 시 히든 엔딩 조건을 "만족도 100 + 특정 수치 100"으로 설정하였으나, 실제 플레이 테스트 결과 8주 안에 해당 조건을 동시에 달성하는 것이 현실적으로 불가능함을 확인했습니다. 또한 배드엔딩 조건을 "만족도 25이하"로 설정하였으나, 실제 플레이 테스트 결과 만족도가 25이하로 떨어지는 것 불가능한 것을 확인했습니다.
  - **문제 해결 과정:** 히든 엔딩 3종의 발동 조건에서 만족도 기준을 완화하여 달성 가능한 수준으로 조정했습니다. (만족도 100 -> 85 + 특정수치 100)  
배드엔딩이 실제로 발생할 수 있도록 조건 수치를 상향 조정했습니다. (만족도 25 이하 -> 35이하)
  - 결과:** 플레이어가 8주 내에 히든 엔딩과 배드엔딩을 실제로 달성할 수 있도록 밸런스 조정을 해두었습니다.

#### 4. 현재까지의 성과 및 결론

- **기대효과 및 활용방안:**
  - 텍스트기반 육성 시뮬레이션을 통해 C++ 객체지향 설계(클래스 분리, 캡슐화, 포인터 관리)를 실습할 수 있는 학습 예제로 활용 가능합니다.
  - 다마고치와 같은 고전 육성 게임의 핵심 메커니즘(수치 관리, 성장 단계, 엔딩 분기)을 콘솔 환경에서 직접 구현해봄으로써 게임 로직 설계 경험을 쌓을 수 있습니다.
  - 향후 GUI 라이브러리나 파일 입출력(세이브/로드 기능)을 추가하는 방향으로 확장 가능한 구조로 설계되었습니다.
- **프로젝트 성과:**
  - GameEntity, GameEngine, ActionManager, UIRenderer, GameManager 5개 클래스의 역할을 명확히 분리하여 객체지향 설계 원칙을 적용한 구조를 완성했습니다.

- 8 주간의 시간 흐름, 주차별 성장 단계 진화, 행동에 따른 수치 변화, 아사/병사/도주/클리어 엔딩 분기 등 기획한 핵심 게임 로직을 모두 구현했습니다.
- UTF-8 인코딩 설정 및 콘솔 코드페이지 변경을 통해 한글 입출력 문제를 해결하고, Windows 콘솔 환경에서 안정적으로 동작하는 실행 파일을 완성했습니다.
- 11가지 엔딩 분기 전체 구현 및 엔딩별 전용 아스키 아트 제작을 완료하였습니다.
- 아이템 인벤토리 시스템(훈련 → 아이템 획득 → 먹이 선택지 해금), 반항 이벤트 시스템(스트레스 50 이상 시 20% 확률 페널티), SlowPrint 텍스트 연출 및 행동별 아스키 아트 전환 UI를 완성하였습니다.

• **향후 개선 방향:**

- 저장 / 불러오기 기능 추가 (파일 입출력)
- GUI 라이브러리 도입
- 인벤토리 전용 UI 화면 구성 및 행동별 아스키 아트 종류 확대
- 난이도 선택 기능 추가 및 엔딩 조건·종류 확장

---

**엔딩 시스템 — 11가지 분기**

|           |                   |                         |
|-----------|-------------------|-------------------------|
| <b>배드</b> | 아사 엔딩             | 포만감 = 0                 |
|           | 병사 엔딩             | 청결도 = 0                 |
|           | 도주 엔딩             | 스트레스 $\geq 90$          |
|           | 아쉬운 이별            | 8주 클리어, 만족도 $\leq 35$   |
| <b>노말</b> | 아쉬운 이별 (방생)       | 만족도 36~50               |
| <b>굿</b>  | 예상치 못한 동거 / 방생 성공 | 만족도 51~75, 50% 랜덤 분기    |
|           | 앞으로도 잘 부탁해        | 만족도 $\geq 76$           |
| <b>히든</b> | 화석이 되어버린 알        | 1주차 행동 5번 모두 방치하기       |
|           | 새이기를 포기한거야?       | 포만감 100 + 만족도 $\geq 85$ |
|           | 너 덕분에 내가 인생 폼났다!  | 훈련도 100 + 만족도 $\geq 85$ |
|           | 조류 잡지 모델이 되었다     | 청결도 100 + 만족도 $\geq 85$ |



## 5. 실제 개발 일정 및 각 팀원 업무 분장

- 업무 분장

- 게임 기획 담당: 이준규

- 기획서 작성 및 보고서, 포스터, PPT 제작
    - **GameEntity (새 객체 클래스)**: 새의 4가지 핵심 수치(포만감, 청결도, 훈련도, 스트레스), 성장 단계 및 상태 확인 로직 구현

- 메인 로직 개발 담당: 최연채

- **Time (게임 흐름 제어 클래스)**: 8주간의 시간 흐름과 일일 행동 횟수, 시간대 변경 로직 구현
    - **GameManager (전체 프로세스 제어 클래스)**: 게임의 메인 루프 설계, 각 클래스 간의 상호작용을 중재하며 전체적인 게임 흐름과 객체의 생명 주기를 관리하는 메인 로직 구현

- 입출력 및 UI 담당: 최승민

- 아스키 아트 제작
    - **UIRenderer (화면 출력 클래스)**: 현재 주차 및 상태 정보 상시 노출, 아스키아트 기반의 외형 변화 출력 기능 구현

- 테스트 및 버그 수정 담당: 유이레

- **ActionManager (행동 및 선택지 클래스)**: 성장 단계별 행동 메뉴 및 수치 변화 연동, 아이템 효과 처리 로직 구현

“최종 마무리 목표에 맞춰 전체 기능 구현 및 테스트를 완료하였습니다.”

---

## 6. 참고문헌

<https://play.google.com/store/apps/details?id=com.fivebyte.lifeisagame&hl=ko>

<https://ko.wikipedia.org/wiki/%EB%8B%A4%EB%A7%88%EA%B3%A0%EC%B9%98>